Distinction Graph Machines: Refine-or-Repair Runtime Learning

Anonymous

Abstract

We introduce Distinction Graph Machines, a runtime-learning model whose state is a growing graph of anchored concepts, movable retrieval centroids, local memories, and pairwise distinctions. The guiding rule is refine when the current concepts are too coarse, and repair when a local concept is adequate but its state is wrong. The theory separates these operations. Minimal refinement is the smallest partition or sigma-algebra update that makes new evidence measurable, and contradictions inside an atom cannot be solved by any predictor measurable with respect to the current information. The value of a refinement is exact: under log loss it is conditional mutual information, and under squared loss it is conditional variance reduction. The concrete data structure realizes refinement by creating anchors and distinction edges that generate a finite anchor-based information structure; movable centroids are retrieval statistics for the trainable soft read path. We give prediction and observation algorithms, sample-split statistically guarded objective-descent guarantees for refine-or-repair, a data-structure complexity theorem, and a language-modeling instance in which hidden-state concepts store next-token memories and produce runtime logit corrections without inference-time backpropagation. The canonical large-model layer is DGM-Embedding, whose detached target is the observed next-token embedding and whose adaptive state is updated by the same sequential forward-only rule used at inference. We also discuss adapter-style residual extensions with explicit target choices. Projection Memory and CPM appear as local repair mechanisms after the relevant distinction structure has been fixed.

Contents

1 Introduction **1**

2 Preliminaries **2**
2.1 Notation . . . 2
2.2 Partitions and measurable predictors . . . 2
2.3 Bayes risk under information . . . 3
2.4 Projection memory notation . . . 3

3 Minimal Information Refinement **3**
3.1 The finite refinement rule . . . 3
3.2 The sigma-algebra form . . . 4
3.3 Contradiction inside an atom . . . 4

4 The Value of a Distinction **4**
4.1 Risk monotonicity . . . 4
4.2 Log loss . . . 5
4.3 Squared loss . . . 5
4.4 Finite empirical scores . . . 5

5 Distinction Graph Machines **6**
5.1 State and graph-generated information . . . 6
5.2 Edge-gated prediction . . . 7
5.3 Refine or repair on a graph . . . 8
5.4 Why graph memory is different from a tree . . . 11
5.5 Canonical local memories . . . 11
5.6 Language modeling instance . . . 12
5.7 Complexity . . . 13

6 DGM Layers for Large-Model Training **13**
6.1 Common layer interface . . . 14
6.2 DGM-Embedding layer . . . 14
6.3 Optional DGM-Adapter layer . . . 15
6.4 Detached write path . . . 15
6.5 Why this can scale . . . 16
6.6 Distributed training . . . 17
6.7 Closed-form adapters . . . 17

7 From Distinctions to Projection Memory **18**
7.1 One-hot partition memory . . . 18
7.2 CPM conflict as missing distinction . . . 18
7.3 The hierarchy . . . 18

8 Technical Overview **19**

9 Computational Checks **23**

10 Conclusion **25**

A Appendix Roadmap **29**

B Limitations **29**

C Related Work **30**

D Minimal Refinement Proofs **31**

E Value of Distinction Proofs **32**

F DGM Algorithm and Projection-Bridge Proofs **34**

1 Introduction

Runtime learning has two different failure modes. Sometimes the model is looking at the right kind of state, but the state is numerically wrong; then the right operation is repair. Sometimes the model is using an information structure that identifies situations whose consequences differ; then no local repair can solve the error. The right operation is refinement.

This paper builds a model around that distinction. A Distinction Graph Machine keeps a growing set of anchored concepts, movable retrieval centroids, local memories, and distinction edges. The anchors define stable graph distinctions, the centroids support adaptive retrieval, and the local memories store what the learner currently believes inside each concept. The edges are binary generators: together they induce a partition of hidden-state space and gate which local memories are compatible with a query. On an input x, an encoder produces $h = \phi_\theta(x)$, the machine retrieves nearby centroids, edge-gates their routing weights, and their local memories predict. When the consequence y is observed, the machine decides whether to create a new anchored concept and distinction edges, or repair the memory and centroid of an existing concept.

The guiding rule is

if the atom is wrong, refine; if the local state is wrong, repair.

Refinement creates a new concept when existing concepts are too coarse. Repair updates a local model inside the existing graph. For a simple classifier, repair can be an exemplar or sufficient-statistic update. For a richer local memory, repair can be Projection Memory or CPM.

This is not only a prototype method. Nearest-neighbor and nearest-centroid methods retrieve similar examples, but they do not refine an explicit information structure after consequence conflicts. Decision trees refine information, but force distinctions into a rooted hierarchy and do not attach repairable local memories to graph concepts. Sparse MoE layers route to trained experts, whereas DGM creates runtime concepts from observed conflicts. Cache and kNN language models retrieve recent contexts, whereas DGM separates redundant evidence, novel evidence, and contradictory evidence through graph-generated distinctions before repairing next-token memories. The distinction from nearby methods is summarized as follows.

Method family	What it has	What it lacks relative to DGM
kNN/cache	retrieval	no refine-or-repair rule and no edge-generated distinctions
Decision tree	refinement	no graph concepts and no local repair memory
MoE	routing	trained experts rather than runtime concept creation
TTT	runtime adaptation	gradient/self-supervised updates, not a distinction graph
Fast weights	dynamic memory	residual writes without concept refinement

The theory explains why the two operations should not be collapsed. If two states lie in the same atom but require different outputs, no measurable predictor over the current partition can satisfy both. This is not an optimization failure; it is a missing distinction. Conversely, if the current concept is adequate and only its local state is stale, creating a new concept is unnecessary. The model should update the local memory.

We formalize both sides. Minimal information refinement is the smallest partition or sigma-algebra update that makes new evidence observable. Its value is exact under standard losses: conditional mutual information for log loss and conditional variance reduction for squared loss. The

DGM data structure adds the engineering layer missing from the pure refinement principle: graph-induced partitions, edge-gated routing, canonical local memories, statistically guarded penalized refine-or-repair rules, language-modeling integration, and train/test APIs. The exact information-structure object is anchor-DGM: frozen anchors and distinction edges define the formal sigma-algebra, while movable centroids are retrieval statistics for the soft read path. The canonical large-model layer is DGM-Embedding. We treat DGM-Adapter as an optional residual extension because it requires an explicit modeling choice for the hidden residual target. In both cases the read path is differentiable and the write path is detached, so that large models can learn to use DGM memory during training without backpropagating through the entire memory history.

Mechanism	Formal claim
Minimal refinement	Smallest information update making evidence measurable
Contradiction	No current measurable predictor resolves conflict inside an atom
Edge generator	Adds a binary distinction between confused anchors
Edge-gated routing	Soft relaxation of hard edge compatibility
Sample-split refinement	Accepted refinement has positive guarded gain
Local repair	Updates statistics without changing the anchor-generated information structure
DGM-Embedding	Forward-only next-token memory correction with detached writes

Organization. Section 2 fixes notation and information structures. Section 3 proves the refinement principle and the contradiction theorem. Section 4 quantifies the value of distinctions. Section 5 defines DGM as a concrete model and algorithm, including a language-modeling instance. Section 6 gives a scalable DGM layer for large-model training. Section 7 relates local repair to Projection Learning and CPM. Section 8 gives proof sketches. Section 9 gives executable checks. Section 10 concludes.

2 Preliminaries

2.1 Notation

All probability spaces are denoted by $(\Omega, \mathcal{A}, \mathbb{P})$ unless the finite case is stated explicitly. Random variables are functions on Ω. For vectors $x \in \mathbb{R}^d$, lengths use the ℓ_2 norm $\|x\|_2$. For matrices, $\|M\|_F$ is the Frobenius norm. Inner products are written $\langle x, y \rangle$. For a matrix K, the Moore-Penrose pseudoinverse is $K^\dagger$, and $KK^\dagger$ is the orthogonal projector onto the span of the columns of K.

For sigma-algebras $\mathcal{F}, \mathcal{G} \subseteq \mathcal{A}$, the inclusion $\mathcal{F} \subseteq \mathcal{G}$ means that $\mathcal{G}$ is at least as informative as $\mathcal{F}$. The join $\mathcal{F} \vee \mathcal{G}$ is the smallest sigma-algebra containing both. For a random variable Z, $\sigma(Z)$ is the sigma-algebra generated by Z; for an event E, $\sigma(E) = \{\varnothing, E, E^c, \Omega\}$.

2.2 Partitions and measurable predictors

In the finite setting, an information structure is a partition

$$\mathcal{P} = \{B_1, \ldots, B_n\}$$

of Ω into nonempty disjoint atoms. A partition $\mathcal{Q}$ refines $\mathcal{P}$ if every atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}$. An event $E \subseteq \Omega$ is $\mathcal{P}$-measurable if it is a union of atoms of $\mathcal{P}$.

A prediction or action $f : \Omega \to A$ is $\mathcal{P}$-measurable if it is constant on each atom:

$$\omega, \omega' \in B \in \mathcal{P} \quad \Longrightarrow \quad f(\omega) = f(\omega').$$

Equivalently, f can depend only on which atom contains the current world state. In the sigma-algebra setting, admissible predictors are $\mathcal{F}$-measurable functions.

2.3 Bayes risk under information

Let $Y : \Omega \to \mathcal{Y}$ be a target random variable, let A be an action space, and let $L : \mathcal{Y} \times A \to \mathbb{R}$ be a loss for which the expectations below are well-defined. The Bayes risk available under information $\mathcal{F}$ is

$$R_L(\mathcal{F}) = \inf\{\mathbb{E}[L(Y, a)] : a : \Omega \to A,\ a \text{ is } \mathcal{F}\text{-measurable}\}. \tag{1}$$

Thus $R_L(\mathcal{F})$ is the best population risk achievable by a predictor that can use only distinctions in $\mathcal{F}$.

2.4 Projection memory notation

For the bridge to Projection Learning, we use a finite partition $\mathcal{P} = \{B_1, \ldots, B_n\}$ and its one-hot feature map $\phi_{\mathcal{P}} : \Omega \to \mathbb{R}^n$:

$$\phi_{\mathcal{P}}(\omega) = e_i \qquad \text{if} \qquad \omega \in B_i.$$

A matrix $W \in \mathbb{R}^{m \times n}$ assigns a value vector to each atom through $W\phi_{\mathcal{P}}(\omega)$. For old key matrices K, a new key k has CPM innovation

$$z = (I - KK^{\dagger})k. \tag{2}$$

3 Minimal Information Refinement

The primitive operation in this paper is refinement. The learner has an information structure and receives evidence that a new event or variable must become observable. The correct update should add that observability and nothing else. This section states the finite theorem first because it is the clearest form of the idea: split only the atoms cut by the evidence.

3.1 The finite refinement rule

Let $\mathcal{P}$ be a finite partition and let $E \subseteq \Omega$ be a new binary distinction. Define

$$\begin{aligned} \mathcal{P}_E = &\{B \cap E : B \in \mathcal{P}, B \cap E \neq \varnothing\} \\ &\cup \{B \cap E^c : B \in \mathcal{P}, B \cap E^c \neq \varnothing\}. \end{aligned} \tag{3}$$

This notation simply keeps the nonempty pieces obtained by cutting each old atom by E.

Theorem 3.1 (Minimal finite refinement)**.** *Let $\mathcal{P}$ be a finite partition of Ω and let $E \subseteq \Omega$. The partition $\mathcal{P}_E$ is the unique minimal partition that refines $\mathcal{P}$ and makes E measurable. More precisely, if $\mathcal{Q}$ refines $\mathcal{P}$ and E is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_E$.*

The theorem gives a canonical update. We do not search over all refined representations; the evidence itself determines the least one. An atom untouched by E is left unchanged. An atom crossed by E is split into the two pieces that the evidence can now distinguish.

3.2 The sigma-algebra form

For general spaces, the same operation is the join. If the current information is $\mathcal{F}_t$ and the evidence is a variable Z_t, the minimal updated information is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(Z_t). \tag{4}$$

Theorem 3.2 (Minimal sigma-algebra refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{A}$ be a sigma-algebra and let Z be a random variable. The join $\mathcal{F} \vee \sigma(Z)$ is the smallest sigma-algebra $\mathcal{G}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable.*

Thus the same object governs finite decision-tree splits and measurable-space refinement. A distinction is not an auxiliary feature added after learning; it is a change in the sigma-algebra of what the learner can condition on.

3.3 Contradiction inside an atom

Minimal refinement is necessary when the current information identifies states that the target separates. This is the simplest obstruction to pure optimization.

Theorem 3.3 (Contradiction requires distinction)**.** *Let $\mathcal{P}$ be a partition of Ω and let $y : \Omega \to A$ be a required output. Suppose ω, ω' lie in the same atom of $\mathcal{P}$ but $y(\omega) \neq y(\omega')$. Then no $\mathcal{P}$-measurable predictor $f : \Omega \to A$ satisfies both $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$.*

The conclusion is structural, not algorithmic. A larger step size, a different optimizer, or more iterations cannot produce two different outputs on states that the representation forces to be the same. The missing operation is not descent. It is refinement.

4 The Value of a Distinction

Refinement increases what the learner can condition on. This does not mean that every distinction is useful for every task. The value of a distinction is measured by the decrease in optimal risk after the information structure is refined.

4.1 Risk monotonicity

The first fact is purely order-theoretic. If $\mathcal{G}$ contains all events in $\mathcal{F}$, then every $\mathcal{F}$-measurable predictor is also $\mathcal{G}$-measurable. The decision class has only grown.

Theorem 4.1 (Bayes-risk monotonicity)**.** *If $\mathcal{F} \subseteq \mathcal{G}$, then for every loss L,*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

This is a population statement. It says that information cannot hurt the best possible decision rule. Statistical estimation, finite samples, and model selection can still overfit; those are algorithmic questions, not failures of the information order.

4.2 Log loss

For a finite target variable Y, log loss asks the learner to output a conditional distribution. The optimal prediction under $\mathcal{F}$ is the conditional law of Y given $\mathcal{F}$, and the optimal risk is conditional entropy.

Theorem 4.2 (Log-loss value)**.** *Let Y be finite and let the loss be $-\log q(Y)$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Thus, for classification and language modeling, a refinement is useful exactly to the extent that it carries conditional mutual information about the consequence beyond what the learner already knows.

4.3 Squared loss

For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss, the optimal predictor under $\mathcal{F}$ is $\mathbb{E}[Y \mid \mathcal{F}]$. Refinement helps precisely when it changes the conditional mean.

Theorem 4.3 (Squared-loss value)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

The identity is the Hilbert-space analogue of the log-loss result. A distinction has squared-loss value when the cells it creates have different conditional means.

4.4 Finite empirical scores

The population identities lead directly to split scores. Suppose an atom B contains samples (x_j, y_j) and a candidate distinction $g : B \to \{0, 1\}$ splits it into B_0 and B_1.

For classification, let $\hat{p}_B$, $\hat{p}_{B_0}$, and $\hat{p}_{B_1}$ be empirical label distributions. The empirical log-loss improvement is

$$\Delta_{\log}(B, g) = |B| H(\hat{p}_B) - |B_0| H(\hat{p}_{B_0}) - |B_1| H(\hat{p}_{B_1}).$$

Equivalently,

$$\Delta_{\log}(B, g) = |B| \widehat{I}(Y; g \mid B).$$

For regression, define

$$\hat{\mu}_B = \frac{1}{|B|} \sum_{j \in B} y_j, \qquad \mathrm{SSE}(B) = \sum_{j \in B} \|y_j - \hat{\mu}_B\|_2^2.$$

The empirical squared-loss improvement is

$$\Delta_2(B, g) = \mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1).$$

It also has the between-cell form

$$\Delta_2(B, g) = |B_0| \, \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1| \, \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

These scores implement the same principle as the population theory: split only when the proposed distinction has measurable consequence value.

5 Distinction Graph Machines

The tree model is a useful finite special case, but it is not the most natural runtime data structure. A tree makes every decision through one irreversible path. A more flexible object is a graph of concepts: local memories live at prototypes, and distinctions live on edges between prototypes that have been confused. The central object is

$$\text{DGM} = \text{graph-generated information structure} + \text{local memory repair}.$$

This section defines that object. The graph is not metadata: its edges generate the finite information structure and gate the read path. The main-text statements below give the operational theorem forms; full proofs are collected in Appendix F.

5.1 State and graph-generated information

Let $\phi_\theta : \mathcal{X} \to \mathbb{R}^d$ be an encoder and write $h = \phi_\theta(x)$. A Distinction Graph Machine has runtime state

$$\mathcal{S}_t = \left(\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t\right), \qquad G_t = (V_t, E_t).$$

Here $a_j \in \mathbb{R}^d$ is a frozen distinction anchor, $c_j \in \mathbb{R}^d$ is a retrieval centroid, M_j is a local memory, n_j is a usage count, and E_t is a set of distinction edges. A new concept is initialized with $a_j = c_j = h$. Subsequent repair may move c_j, but anchors are fixed once created, so old distinctions keep their original semantics. An edge $e = (i, j)$ stores a linear distinction

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j),$$

with side labels $s_{e,i} = +1$ and $s_{e,j} = -1$. Its binary generator is

$$g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}.$$

The edge set induces an information structure. Define the graph signature

$$\sigma_t^G(h) = \big(g_e(h)\big)_{e \in E_t} \in \{0,1\}^{E_t}.$$

Two hidden states are indistinguishable by the current graph if their signatures agree. The nonempty level sets

$$A_a^t = \{h : \sigma_t^G(h) = a\}, \qquad a \in \{0,1\}^{E_t},$$

form a finite partition $\mathcal{P}_t^G$ of hidden-state space, and the generated sigma-algebra is $\sigma(\mathcal{P}_t^G)$. Adding a new edge is therefore a join with one binary generator:

$$\mathcal{P}_{t+1}^G = \mathcal{P}_t^G \vee \sigma(g_e).$$

Anchors generate the distinction skeleton. Centroids and local memories are adaptive statistics attached to atoms or sparse neighborhoods of this graph-generated partition.

Theorem 5.1 (Prototype distinction separates its endpoints)**.** *If $a_i \neq a_j$ and u_e, b_e are defined as above for edge $e = (i, j)$, then*

$$u_e^\top a_i - b_e = \frac{1}{2}\|a_i - a_j\|_2, \qquad u_e^\top a_j - b_e = -\frac{1}{2}\|a_i - a_j\|_2.$$

Thus each edge distinction separates the two anchors that caused it.

Theorem 5.2 (DGM graph generates a finite information structure)**.** *For any finite edge set E_t, equality of signatures $\sigma_t^G(h)$ is an equivalence relation whose classes form a finite partition $\mathcal{P}_t^G$. Adding one edge g refines the partition by $\mathcal{P}_t^G \vee \sigma(g)$.*

5.2 Edge-gated prediction

Given h, the soft implementation first computes retrieval distances $d_j(h) = \|h - c_j\|_2^2$ to movable centroids and a sparse neighborhood

$$N_k(h) \subseteq \{1, \dots, N_t\}.$$

If $I_t(j)$ is the set of edges incident to prototype j, define the soft compatibility of h with prototype j by

$$\log \chi_j(h) = \frac{1}{\sqrt{\max\{1, |I_t(j)|\}}} \sum_{e \in I_t(j)} \beta_e \log \operatorname{sigmoid}\left(\frac{s_{e,j}(u_e^\top h - b_e)}{\tau_e}\right),$$

where structural distinction edges have $\beta_e \in (0, 1]$ and the empty sum gives $\chi_j(h) = 1$. The degree normalization avoids systematically suppressing high-degree prototypes while preserving hard rejection in the low-temperature limit. The edge-gated routing weights are

$$\alpha_j(h) = \frac{\exp(-d_j(h)/T)\chi_j(h)}{\sum_{r \in N_k(h)} \exp(-d_r(h)/T)\chi_r(h)}, \qquad j \in N_k(h). \tag{5}$$

The prediction is the local mixture

$$\hat{y} = \sum_{j \in N_k(h)} \alpha_j(h) M_j(h).$$

The hard version makes the information-structure connection explicit. A prototype j is eligible for h if every incident edge bit places h on j's recorded side:

$$\begin{cases} g_e(h) = 1, & s_{e,j} = +1, \\ g_e(h) = 0, & s_{e,j} = -1, \end{cases} \qquad \text{for every } e \in I_t(j).$$

This convention assigns boundary points to the nonpositive side because $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$. The exact Hard-DGM object used in the refinement theory routes to the nearest eligible anchor, falling back to the nearest anchor if no eligible prototype is available. Soft-DGM in (5) is the trainable read path: it may retrieve by movable centroids, but those centroids are retrieval statistics rather than formal information atoms.

The distinction partition alone is not the whole exact hard route. Anchor distances add a stable order geometry. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be the finite sigma-algebra generated by all pairwise anchor-distance comparison events

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}, \qquad 1 \le i, j \le N_t,$$

with deterministic tie-breaking. The hard routing skeleton is measurable with respect to

$$\mathcal{F}_t^{\text{DGM}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

Thus the exact information-structure theory applies to anchor-DGM. Soft-DGM keeps the same graph skeleton but may use movable centroids and smooth gates for locality, so it should be read as a differentiable engineering relaxation of the anchor-generated information structure rather than as a new formal refinement.

Theorem 5.3 (Anchor-DGM routing is graph-plus-anchor-order measurable)**.** *With finite E_t, finite frozen anchors $a_1, \ldots, a_{N_t}$, and deterministic tie-breaking, the exact Hard-DGM selected prototype index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\mathrm{DGM}} = \mathcal{F}_t^{\mathrm{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.*

The next statement is analytic rather than structural: it applies to whichever fixed distances a soft read uses. With anchor distances it recovers the exact hard anchor route; with centroid distances it describes the behavior of the soft retrieval approximation.

Theorem 5.4 (Soft routing approaches hard routing)**.** *If the hard nearest eligible prototype is unique with distance margin $\delta > 0$, then its soft routing weight tends to one exponentially as $T \to 0$. If every wrong-side candidate violates some incident structural edge with weight at least $\beta_{\min} > 0$, signed margin at least $\delta_e > 0$, and degree at most $D_{\max}$, then each wrong-side compatibility decays uniformly after degree normalization. If at least one eligible reference candidate has compatibility bounded below and the wrong-side/reference distance-score ratios are bounded, then total wrong-side routing mass vanishes as the edge temperature $\tau_e \to 0$.*

Algorithm 1 PREDICT for a Distinction Graph Machine

```
Input: state S_t = ({a_j, c_j, M_j, n_j}_{j=1}^{N_t}, G_t), encoder φ_θ, example x
h ← φ_θ(x)
N_k(h) ← the k nearest centroids to h
for j ∈ N_k(h) do
    compute edge compatibility χ_j(h) from incident distinctions
    compute edge-gated weight α_j(h) by (5)
end for
return Σ_{j∈N_k(h)} α_j(h) M_j(h)
```

5.3 Refine or repair on a graph

After observing consequence y, DGM first asks whether an existing concept can absorb the evidence. Let $j^\star$ be the highest-weight prototype under (5). Define local surprise

$$\rho_t = L(y, M_{j^\star}(h)).$$

Define neighborhood consequence disagreement on $N_k(h)$ by the task-appropriate local prediction divergence. For classification one may use

$$D_t = \max_{i,r \in N_k(h)} \mathrm{TV}(p_i(\cdot \mid h), p_r(\cdot \mid h)),$$

where p_j is the distribution represented by memory M_j. For regression one may use

$$D_t = \max_{i,r \in N_k(h)} \|\mu_i(h) - \mu_r(h)\|_2 ,$$

and for language modeling the log-loss surprise $-\log p_{j^\star}(x_{t+1} \mid x_{\leq t})$ is the local disagreement signal.

If ρ_t is small, the information structure is treated as adequate and the machine repairs the local state:

$$M_{j^\star} \leftarrow \mathrm{Repair}(M_{j^\star}, h, y), \qquad c_{j^\star} \leftarrow c_{j^\star} + \eta_c(h - c_{j^\star}), \qquad n_{j^\star} \leftarrow n_{j^\star} + 1.$$

The anchor $a_{j^\star}$ is not moved by repair. Consequently, local memory repair and centroid adaptation do not rewrite previously accepted edge generators. If local surprise and disagreement are large, DGM evaluates a candidate refinement. To match the statistical guarantee, the local evidence is logically split into a proposal buffer and a scoring buffer,

$$\mathcal{B}_t^{\text{prop}} \qquad \text{and} \qquad \mathcal{B}_t^{\text{score}}.$$

The proposal buffer, together with the current observation, may be used to propose a new concept and confused-neighbor edges. The scoring buffer is held out from proposal generation and is used only to estimate the refinement gain. A practical implementation can maintain the split by reservoir sampling incoming observations into the two buffers. If an implementation reuses one buffer for both proposal and scoring, the empirical rule still defines a heuristic, but Theorem 5.7 applies to the sample-split version.

A candidate creates

$$a_{\text{new}} = h, \qquad c_{\text{new}} = h, \qquad M_{\text{new}} = \text{InitMemory}(h, y),$$

and adds q distinction edges from the new anchored concept to the confused neighbors. Let $\widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}})$ be the empirical local loss on the scoring buffer after routing with state $\mathcal{S}$. The refinement gain is

$$\Delta_{\text{refine}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_t^{\text{new}}; \mathcal{B}_t^{\text{score}}).$$

The candidate is accepted only if

$$\Delta_{\text{refine}} > \lambda_{\text{proto}} + \lambda_{\text{edge}} q + \tau + 2\epsilon_m \qquad \text{and} \qquad D_t > \tau_{\text{disagree}}. \tag{6}$$

Here ϵ_m is a statistical safety margin for a scoring buffer of size $m = |\mathcal{B}_t^{\text{score}}|$; it can be set to zero for a purely empirical implementation, or to the finite-class bound in Theorem 5.7. Otherwise DGM performs the best local repair available under the current graph. This makes refinement a statistically guarded penalized empirical-risk decision rather than an unscored heuristic.

Algorithm 2 OBSERVE for a Distinction Graph Machine

Input: state $\mathcal{S}_t$, labeled consequence (x, y)
$h \leftarrow \phi_\theta(x)$
if $N_t = 0$ **then**
  create anchor and centroid $a_1 = c_1 = h$ with memory $\text{InitMemory}(h, y)$
  return updated state
end if
retrieve $N_k(h)$ and compute edge-gated routing weights
$j^\star \leftarrow \text{argmax}_{j \in N_k(h)} \alpha_j(h)$
$\rho \leftarrow L(y, M_{j^\star}(h))$
compute neighborhood disagreement D_t
if $\rho \leq \tau_{\text{repair}}$ **then**
  repair $M_{j^\star}$ and move centroid $c_{j^\star}$ toward h; keep anchor $a_{j^\star}$ fixed
else
  form a candidate new anchor/centroid from h and $\mathcal{B}_t^{\text{prop}}$, with candidate edges to confused neighbors
  estimate Δ_{refine} on the held-out scoring buffer $\mathcal{B}_t^{\text{score}}$
  if $\Delta_{\text{refine}} > \lambda_{\text{proto}} + \lambda_{\text{edge}} q + \tau + 2\epsilon_m$ and $D_t > \tau_{\text{disagree}}$ **then**
    commit the new anchored concept and its distinction edges
  else
    repair the best existing local memory under the current graph
  end if
end if
return updated state

Definition 5.5 (Monotone local repair)**.** *A repair operator is monotone on scoring buffer $\mathcal{B}_t^{\text{score}}$ if the state $\mathcal{S}_{t+1}$ obtained by repairing local memories and moving retrieval centroids, while keeping anchors and edges fixed, satisfies*

$$\widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) \leq \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}).$$

Theorem 5.6 (Refine-or-repair objective descent)**.** *Let the local scoring objective on a fixed scoring buffer $\mathcal{B}_t^{\text{score}}$ be*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{proto}} N_{\text{proto}}(\mathcal{S}) + \lambda_{\text{edge}} |E(\mathcal{S})|.$$

If repair is monotone on the scoring buffer, then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement adding one anchored concept and q edges is accepted only under (6), then the empirical penalized objective $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\tau + 2\epsilon_m$.

This is a local-buffer statement. It does not imply monotone descent of a global objective when the scoring buffer changes over time; a global finiteness claim requires a common potential or validation objective.

Theorem 5.7 (Statistically safe finite-class refinement)**.** *Assume losses lie in $[0, 1]$, the scoring buffer has m independent samples, and the finite candidate refinement class $\mathcal{H}$ is fixed before scoring, or is generated from $\mathcal{B}_t^{\text{prop}}$ and the current observation, independent of $\mathcal{B}_t^{\text{score}}$. With probability at least $1 - \delta$, empirical and population risks are uniformly within*

$$\epsilon_m = \sqrt{\frac{\log(2(|\mathcal{H}| + 1)/\delta)}{2m}}.$$

Consequently every empirical gain $\widehat{\Delta}(g)$ differs from its population gain $\Delta(g)$ by at most $2\epsilon_m$. Hence an accepted refinement satisfying

$$\widehat{\Delta}(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_m$$

has positive population gain after paying complexity cost and margin τ.

Theorem 5.8 (Anchor-stable DGM information structure)**.** *Let*

$$\mathcal{F}_t^{\mathrm{DGM}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\} : 1 \le i, j \le N_t\Big).$$

If a repair step changes only retrieval centroids c_j, local memories M_j, and counts n_j, while keeping anchors a_j and edges E_t fixed, then $\mathcal{F}_{t+1}^{\mathrm{DGM}} = \mathcal{F}_t^{\mathrm{DGM}}$. If a refinement adds a new anchor and new edge generators, then the updated information structure is the join of $\mathcal{F}_t^{\mathrm{DGM}}$ with the new anchor-order events and the new edge generators.

Corollary 5.9 (Finite number of accepted refinements)**.** *If there is a common nonnegative potential $\Phi(\mathcal{S})$ and every accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$. In particular, this applies to a fixed scoring buffer or validation objective. The local-buffer descent theorem alone should not be read as a global finiteness result when the scoring buffer changes over time.*

5.4 Why graph memory is different from a tree

A tree stores distinctions as a hierarchy. A mistake near the root changes every downstream routing decision. A graph stores distinctions locally between concepts. Creating a new concept does not rewrite old paths; it adds a local alternative and edges explaining why the new concept differs from its neighbors.

Proposition 5.10 (Trees as generalized hard-DGMs)**.** *Given a binary distinction tree, construct one generalized hard-DGM prototype for each leaf and attach the path distinctions as hard eligibility constraints for that prototype. If generalized hard-DGM uses exact hard routing to the unique eligible leaf prototype and each prototype memory equals the tree leaf predictor, then generalized hard-DGM and the tree make identical predictions.*

Thus the distinction-graph view contains trees after allowing arbitrary prototype-level eligibility constraints. A decision tree is the sparse case in which distinctions are forced into a rooted hierarchy and routing is hard and single-path.

5.5 Canonical local memories

The local memory M_j is the repair mechanism inside an already chosen concept. To keep the main method fixed, we use two canonical variants.

DGM-Mean. For classification or language modeling with output embeddings $E[y] \in \mathbb{R}^{d_e}$, prototype j stores

$$r_j = \frac{1}{n_j} \sum_{s \in C_j} E[y_s].$$

It contributes the score

$$\ell_{\mathrm{DGM}}(y \mid h) = \sum_{j \in N_k(h)} \alpha_j(h) E[y]^\top r_j.$$

Repair is the sufficient-statistic update

$$r_j^+ = \frac{n_j r_j + \alpha_j(h) E[y]}{n_j + \alpha_j(h)}, \qquad n_j^+ = n_j + \alpha_j(h).$$

DGM-Ridge. For few-shot adaptation or local readout repair, a memory stores a small support set $Z \in \mathbb{R}^{n \times d}$ and labels $Y \in \mathbb{R}^{n \times C}$. The closed-form adapter is

$$\Gamma = (ZZ^\top + \lambda I)^{-1} Y, \qquad \mathrm{logits}(z) = z Z^\top \Gamma.$$

This is the canonical stronger adapter version. Projection Memory and CPM are advanced repair rules inside M_j after the graph has fixed the relevant local concept; Section 7 treats that relation directly.

5.6 Language modeling instance

For language modeling, the world state is a prefix and the consequence is the next observed token. Let a frozen or slowly trained backbone produce hidden states

$$h_t = \phi_\theta(x_{\leq t}).$$

DGM is inserted as a runtime memory over hidden states. Each anchor-centroid pair (a_j, c_j) represents a recurring context type: a citation pattern, a local syntax regime, a document-specific entity usage, or a repeated task format. Distinction edges separate contexts whose next-token consequences force a split. The DGM-Mean memory predicts a correction to the next-token distribution:

$$\ell_{\mathrm{DGM}}(y \mid h_t) = \sum_{j \in N_k(h_t)} \alpha_j(h_t) E[y]^\top r_j,$$

where α_j is edge-gated. The final logits are

$$\ell(y \mid x_{\leq t}) = \ell_\theta(y \mid x_{\leq t}) + \lambda_t \ell_{\mathrm{DGM}}(y \mid h_t).$$

Here λ_t is a learned or fixed trust weight. If the LM head uses tied embeddings and r_j lives in the hidden/output-embedding space, the same correction can be computed as $h_t \leftarrow h_t + \lambda_t \sum_j \alpha_j(h_t) r_j$ before the usual LM-head multiplication, avoiding a second full-vocabulary projection. After the next token x_{t+1} is observed, the machine updates by Observe with consequence $y = x_{t+1}$.

This gives a direct test-time learning protocol for language modeling. During streaming prediction, document continuation, user correction, tool feedback, or supervised next-token evaluation, the next observed token is genuine evidence and can refine or repair the runtime memory. During unconstrained generation, the model should not update from its own sampled token by default, because self-generated samples are actions, not external consequences.

This differs from a cache language model in what is updated. A cache appends or retrieves recent context-token pairs. DGM updates a graph of runtime concepts: redundant observations repair a local next-token memory, while surprising consequences can create a new anchored concept and edges that gate future retrieval.

Algorithm 3 ObserveNextToken for DGM language modeling

Input: prefix $x_{\leq t}$, observed token x_{t+1}, DGM state $\mathcal{S}_t$
$h_t \leftarrow \phi_\theta(x_{\leq t})$
retrieve $N_k(h_t)$ and compute edge-gated logits $\ell_\theta + \lambda_t \ell_{\text{DGM}}$
$j^\star \leftarrow$ highest-weight prototype
$\rho \leftarrow -\log p(x_{t+1} \mid x_{\leq t})$
if $\rho \leq \tau_{\text{repair}}$ **then**
    update the local token memory at $j^\star$
else
    accept a new anchored concept only if the penalized refinement gain test holds
end if
return updated state

5.7 Complexity

Let

$$Q_k(h) = \sum_{j \in N_k(h)} |I_t(j)|$$

be the number of incident edge gates evaluated for the selected candidates. With exact nearest-neighbor search, prediction costs

$$O(N_t d + Q_k(h) + kd + C_{\text{local}}),$$

where C_{local} is the cost of evaluating the selected local memories. With approximate nearest-neighbor indexing, the $N_t d$ term can be replaced by the query cost of the index. Creating an anchor-centroid pair costs $O(d)$ plus the cost of initializing its memory. Adding q distinction edges costs $O(qd)$. If each prototype stores an embedding mean in $\mathbb{R}^{d_e}$, runtime memory is

$$O(N_t(2d + d_e) + |E_t|d).$$

Theorem 5.11 (DGM data-structure bound)**.** *For exact search over N_t centroids in $\mathbb{R}^d$, one edge-gated DGM prediction costs $O(N_t d + Q_k + kd + C_{\text{local}})$. One observation that repairs a selected prototype costs this plus the local repair cost. One observation that creates a new anchor-centroid pair and q distinction edges costs an additional $O(d + qd)$. With embedding-mean memories, the runtime state uses $O(N_t(2d + d_e) + |E_t|d)$ memory.*

6 DGM Layers for Large-Model Training

The data structure above is useful as a runtime learner, but a large model needs a layer. The layer must satisfy two constraints that pull in opposite directions. Its read computation must be differentiable, so that the backbone can learn to use the memory. Its write computation must be cheap and stable, so that training and inference can use the same update rule without backpropagating through the entire memory history.

6.1 Common layer interface

Let $h_t \in \mathbb{R}^d$ be the hidden state at a Transformer layer and let

$$q_t = Qh_t \in \mathbb{R}^{d_q}$$

be the DGM query. A DGM layer maintains frozen query-space anchors $a_j \in \mathbb{R}^{d_q}$, movable query-space centroids $c_j \in \mathbb{R}^{d_q}$, local memories, counts, and distinction edges. The formal information structure is generated by anchors and edges. Retrieval uses the centroids as adaptive statistics, while edges are computed from anchors. Retrieval gives $N_k(q_t)$ and the read path uses the edge-gated weights from (5), with q_t in place of h. The read path is differentiable through the selected memories, routing weights, and backbone parameters. The top-k retrieval itself may be treated as a sparse selection operator, as in mixture-of-experts and retrieval systems.

6.2 DGM-Embedding layer

The simplest large-model layer stores an output embedding mean at each prototype:

$$\mathcal{S}_t^{\text{emb}} = \{a_j, c_j, r_j, n_j\}_{j=1}^{N_t} \cup G_t, \qquad r_j \in \mathbb{R}^{d_e}.$$

For language modeling, with output embedding $E[y]$, the DGM read contributes a logit correction

$$\Delta\ell_t(y) = \sum_{j \in N_k(q_t)} \alpha_j(q_t) E[y]^\top r_j.$$

Equivalently, define the mixed memory vector

$$r_{\text{mix}}(q_t) = \sum_{j \in N_k(q_t)} \alpha_j(q_t) r_j.$$

Then $\Delta\ell_t(y) = E[y]^\top r_{\text{mix}}(q_t)$. With tied output embeddings and $d_e = d$, this can be implemented as a hidden-state correction

$$h_t^{\text{DGM}} = h_t + \lambda_t r_{\text{mix}}(q_t),$$

followed by the usual LM-head multiplication. This avoids an extra vocabulary-sized projection. The model adds this to the backbone logits:

$$\ell(y \mid x_{\leq t}) = \ell_\theta(y \mid x_{\leq t}) + \lambda_t \Delta\ell_t(y).$$

After the next token is observed, $v_t = E[x_{t+1}]$ gives the detached sufficient-statistic write

$$r_j^+ = \frac{n_j r_j + \alpha_j(q_t) v_t}{n_j + \alpha_j(q_t)}, \qquad n_j^+ = n_j + \alpha_j(q_t).$$

If the penalized refinement test in (6) accepts a new concept, the layer creates a new anchor and centroid at q_t, initializes $r_{\text{new}} = v_t$, and adds distinction edges to the confused prototypes. Ordinary repairs may update centroids and memories but leave anchors fixed.

6.3 Optional DGM-Adapter layer

The DGM-Embedding layer is the canonical layer because its target is the observed next-token embedding. A richer optional layer stores a local linear repair map:

$$\mathcal{S}_t^{\text{adp}} = \{a_j, c_j, W_j, n_j\}_{j=1}^{N_t} \cup G_t, \qquad W_j \in \mathbb{R}^{d\times r}.$$

Let $K : \mathbb{R}^d \to \mathbb{R}^r$ be a learned or fixed key projection and set $k_t = Kh_t$. The adapter read is

$$\Delta h_t = \sum_{j\in N_k(q_t)} \alpha_j(q_t) W_j k_t, \qquad \tilde{h}_t = h_t + \gamma_\theta(h_t)\Delta h_t.$$

Here $W_j k_t$ is a hidden-space residual, so the write rule requires an explicit residual target $v_t \in \mathbb{R}^d$. Three natural choices are:

$$\begin{aligned}
v_t &= PE[x_{t+1}] && \text{token-embedding target, with } P : \mathbb{R}^{d_e} \to \mathbb{R}^d,\\
v_t &= P\left(E[x_{t+1}] - \hat{e}_t\right) && \text{teacher-forced output-error target,}\\
v_t &= h_t^{\text{teacher}} - h_t && \text{distillation or future-state residual target.}
\end{aligned}$$

The first choice reduces the adapter to an embedding-memory variant when P is the identity. The second requires a learned projection from output-embedding error to hidden residual space. The third requires an external teacher or delayed target state. Without one of these choices, the Adapter layer is only an architectural extension, not the main method.

Given such a local residual target, the projected rank-one repair for a selected prototype is

$$W_j^+ = W_j + \frac{\eta\alpha_j(q_t)}{1 + \eta\alpha_j(q_t)\,\|k_t\|_2^2}\big(v_t - W_j k_t\big)k_t^\top.$$

Equivalently, a memory may maintain small ridge sufficient statistics and solve the local adapter in closed form. If the implementation uses a low-rank factorization $W_j = B_j A_j$ for the read path, the closed-form write is a repair of W_j or its sufficient statistics; it is not a simultaneous closed-form update of arbitrary factors A_j and B_j.

6.4 Detached write path

The update is detached:

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

In the sequential training mode below, the model trains slow parameters through the read path and learns fast state through the same observation rule used at inference. The layer is not a replacement for backpropagation through the backbone. It is a replacement for inference-time and history-time backpropagation through the adaptive state.

Algorithm 4 SequentialDetachedTrainStep for a DGM Transformer layer

Input: token block $x_{1:L}$, slow parameters Θ, DGM state $\mathcal{S}$
$\mathcal{L} \leftarrow 0$
for $t = 1, \ldots, L$ **do**
  compute hidden state h_t using the backbone
  $q_t \leftarrow Qh_t$
  retrieve $N_k(q_t)$ from the prototype bank
  compute edge-gated weights $\alpha_j(q_t)$
  apply the DGM-Embedding read, or an optional DGM-Adapter read with a defined target protocol
  produce logits for x_{t+1} and accumulate $\mathcal{L} \leftarrow \mathcal{L} + \ell_t$
  $\mathcal{S} \leftarrow \mathrm{stopgrad}(\mathrm{DGMObserve}(\mathrm{stopgrad}(\mathcal{S}), h_t, x_{t+1}))$
end for
update slow parameters Θ by backpropagation through the accumulated selected read computations
return updated Θ and $\mathcal{S}$

Block-lagged approximation. For throughput, an implementation may instead hold $\mathcal{S}$ fixed during the forward pass for a whole block, backpropagate through that block, and then apply detached DGMObserve updates for $t = 1, \ldots, L$. This block-lagged DGM training is more parallel, but it is an approximation: token $t + 1$ in the block does not see the state update produced after token t. It therefore introduces a train/inference mismatch and should not be described as the same observation program as streaming inference.

6.5 Why this can scale

During sequential detached training, backpropagation sees the selected read computation, while the DGM state is updated in the same order as streaming inference:

$$\text{predict } x_{t+1} \rightarrow \text{observe } x_{t+1} \rightarrow \text{update state} \rightarrow \text{predict } x_{t+2}.$$

Under this mode, the train and inference programs differ only in whether slow parameters receive gradients. Under block-lagged training, this statement is no longer exact; the benefit is parallel block execution.

Theorem 6.1 (DGM layer cost)**.** *Consider a DGM-Adapter layer with hidden dimension d, query dimension d_q, local rank r, top-k retrieval, selected incident edge-gate count Q_k, and prototype-bank query cost $C_{\mathrm{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\mathrm{ret}}(N_t, d_q) + Q_k + kdr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\mathrm{ret}}(N_t, d_q) + Q_k + k(d_q + dr)).$$

For a block of length L, the DGM state history need not be stored for backpropagation; the additional activation memory is the selected read computation, not the full sequence of memory states. The DGM-Embedding layer replaces the kdr adapter term by the selected embedding-logit cost.

The important comparison is with test-time gradient adaptation. A TTT-style layer typically performs an inner backward pass at inference time. A DGM layer performs retrieval, sparse local read, and closed-form or sufficient-statistic repair. The cost is forward-only:

$$\text{DGM inference} = \text{backbone forward} + \text{retrieval} + \text{sparse local repair}.$$

6.6 Distributed training

The prototype bank can be sharded by the same principle used in retrieval and sparse expert systems. Each device owns a subset of prototypes. A query is routed to a small set of shards, each shard returns local top-k candidates, and a final top-k merge selects the memories used by the read path. Only selected memories receive gradients. Detached writes are sent back to the owning shards.

Budgeting is necessary for long training. A practical DGM layer maintains

$$N_t \leq N_{\max}.$$

We use the following default budget policy. Each prototype tracks last use time ℓ_j and a usage-decayed score

$$s_j = n_j \exp(-\lambda_{\text{age}}(t - \ell_j)).$$

When a refinement would exceed budget, the layer first tries a compatible merge. Prototypes i, j are merge candidates only if

$$\|c_i - c_j\|_2 < \epsilon_c, \qquad D(M_i, M_j) < \epsilon_M,$$

and there is no active high-confidence distinction edge separating them. The merged prototype keeps the higher-score anchor, uses the usage-weighted centroid, and merges sufficient statistics for memories that support additive statistics. If no compatible merge is available, the layer evicts the unprotected prototype with the smallest s_j and removes its incident low-confidence edges. Prototypes incident to high-confidence edges are protected; if all prototypes are protected, the layer rejects the proposed refinement or compresses the lowest-score memory into an archive.

These budget actions are maintenance operations, not learning operations in the formal refine-or-repair sense. The refine-or-repair theorems apply between maintenance events. Merge, eviction, and compression are resource-management approximations; they may destroy information and therefore should not be counted as refinement steps.

6.7 Closed-form adapters

DGM-Ridge is the main closed-form adapter used in our checks. Let $Z \in \mathbb{R}^{n \times d}$ be normalized support embeddings stored by a DGM memory and let $Y \in \mathbb{R}^{n \times C}$ be one-hot consequences. The adapter computes

$$\Gamma = (ZZ^\top + \lambda I)^{-1} Y, \qquad \text{logits}(z) = zZ^\top \Gamma.$$

This is a projection-style repair: it fits the local adapter in one linear solve rather than by iterated backpropagation. When n is small, as in few-shot adaptation or a retrieved local neighborhood, the solve is cheap. In a large model, the same idea can be applied per prototype, per document, or per retrieved shard.

7 From Distinctions to Projection Memory

DGM uses refinement and repair in the same runtime object. Distinction Learning asks what the learner can tell apart. Projection Learning assumes such a representation and asks how a state inside it should be minimally repaired. This section makes the relation formal in the finite one-hot case.

7.1 One-hot partition memory

Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ and let $\phi_{\mathcal{P}}$ be the one-hot feature map from Section 2. A memory matrix $W \in \mathbb{R}^{m \times n}$ defines a value map

$$f_W(\omega) = W\phi_{\mathcal{P}}(\omega).$$

This is exactly a table over the atoms of $\mathcal{P}$: every state inside the same atom receives the same value.

Theorem 7.1 (One-hot memory is partition-measurable)**.** *For every $W \in \mathbb{R}^{m \times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is represented by some W through $f = f_W$.*

Thus a one-hot associative memory is not merely analogous to a partition. It is an exact finite representation of all vector-valued predictors measurable with respect to that partition.

7.2 CPM conflict as missing distinction

Conservative Projection Memory detects conflict when a new key has zero innovation but nonzero residual. In the one-hot finite representation, this diagnostic is exactly the statement that two world states have been placed in the same atom despite requiring different values.

Theorem 7.2 (CPM conflict equals missing distinction)**.** *Let $\omega, \omega' \in B_i$ for an atom $B_i \in \mathcal{P}$, and suppose required values satisfy $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If $We_i = v(\omega)$ has already been committed, then evidence requiring $We_i = v(\omega')$ has CPM innovation $z = 0$ and residual $e \neq 0$.*

The theorem explains why a projection update should stop. The key is not new; the old information structure maps both states to the same coordinate. The residual is nonzero because the consequence differs. The missing update is a refinement of $\mathcal{P}$ that separates ω from ω'.

7.3 The hierarchy

The resulting hierarchy is

$$\text{distinction refinement} \supset \text{projection repair} \supset \text{implicit gradient} \supset \text{explicit gradient descent.}$$

Distinction refinement changes the sigma-algebra $\mathcal{F}_t$. Projection repair changes a state s_t inside a fixed information geometry. Gradient descent is the linearized ℓ_2 special case of soft projection. Backpropagation is the mechanism that computes such gradients in differentiable static graphs.

This is not a rejection of gradient learning or projection memory. It is a separation of roles. When the representation is adequate, repair and optimization are the right tools. When the representation identifies states that consequences require it to separate, the first learning act is to create the missing distinction.

8 Technical Overview

This section gives the proof ideas in the same order as the main text. Full formal versions appear in Appendices D, E, and F.

Theorem 8.1 (Minimal finite refinement, proof sketch for Theorem D.1)**.** *The main-text statement is Theorem 3.1.*

Proof sketch. Every atom of $\mathcal{P}_E$ lies inside an atom of $\mathcal{P}$, so $\mathcal{P}_E$ refines $\mathcal{P}$. Also

$$E = \bigcup_{B \in \mathcal{P}} (B \cap E),$$

so E is $\mathcal{P}_E$-measurable. If $\mathcal{Q}$ refines $\mathcal{P}$ and makes E measurable, then every atom $Q \in \mathcal{Q}$ satisfies

$$Q \subseteq B$$

for some $B \in \mathcal{P}$, and also either $Q \subseteq E$ or $Q \subseteq E^c$. Hence

$$Q \subseteq B \cap E \qquad \text{or} \qquad Q \subseteq B \cap E^c.$$

Thus $\mathcal{Q}$ refines $\mathcal{P}_E$. □

Theorem 8.2 (Minimal sigma-algebra refinement, proof sketch for Theorem D.2)**.** *The main-text statement is Theorem 3.2.*

Proof sketch. By definition,

$$\mathcal{F} \vee \sigma(Z) = \sigma(\mathcal{F} \cup \sigma(Z)).$$

This sigma-algebra contains $\mathcal{F}$ and makes Z measurable. If $\mathcal{G}$ also contains $\mathcal{F}$ and makes Z measurable, then

$$\mathcal{F} \cup \sigma(Z) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(Z)) \subseteq \mathcal{G}.$$

□

Theorem 8.3 (Contradiction, proof sketch for Theorem D.3)**.** *The main-text statement is Theorem 3.3.*

Proof sketch. If f is $\mathcal{P}$-measurable and ω, ω' lie in the same atom, then

$$f(\omega) = f(\omega').$$

If $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$, then

$$y(\omega) = y(\omega'),$$

contradicting the hypothesis. □

Theorem 8.4 (Risk monotonicity, proof sketch for Theorem E.1)**.** *The main-text statement is Theorem 4.1.*

Proof sketch. If $\mathcal{F} \subseteq \mathcal{G}$, then every $\mathcal{F}$-measurable action is $\mathcal{G}$-measurable. Therefore

$$\{a : a \text{ is } \mathcal{F}\text{-measurable}\} \subseteq \{a : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Taking the infimum of the same expected loss over the larger set gives

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

□

Theorem 8.5 (Log-loss value, proof sketch for Theorem E.2)**.** *The main-text statement is Theorem 4.2.*

Proof sketch. For any $\mathcal{F}$-measurable conditional distribution q,

$$\mathbb{E}[-\log q(Y)] = H(Y \mid \mathcal{F}) + \mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}) \| q(\cdot))\right].$$

The minimizer is $q^\star = \mathbb{P}(Y \in \cdot \mid \mathcal{F})$, so $R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F})$. Similarly $R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G})$. Therefore

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

□

Theorem 8.6 (Squared-loss value, proof sketch for Theorem E.3)**.** *The main-text statement is Theorem 4.3.*

Proof sketch. The optimal $\mathcal{F}$-predictor is $\mathbb{E}[Y \mid \mathcal{F}]$. Since $\mathcal{F} \subseteq \mathcal{G}$,

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = (Y - \mathbb{E}[Y \mid \mathcal{G}]) + (\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]).$$

The two terms are orthogonal because the first has conditional mean zero given $\mathcal{G}$ and the second is $\mathcal{G}$-measurable. Taking squared ℓ_2 norms and expectations gives the identity. □

Theorem 8.7 (Prototype edge separation, proof sketch for Theorem F.1)**.** *The main-text statement is Theorem 5.1.*

Proof sketch. With $u = (a_i - a_j)/\|a_i - a_j\|_2$ and $b = u^\top(a_i + a_j)/2$,

$$u^\top a_i - b = \frac{1}{2} u^\top (a_i - a_j) = \frac{1}{2}\|a_i - a_j\|_2,$$
$$u^\top a_j - b = -\frac{1}{2} u^\top (a_i - a_j) = -\frac{1}{2}\|a_i - a_j\|_2.$$

□

Theorem 8.8 (DGM graph partition, proof sketch for Theorem F.2)**.** *The main-text statement is Theorem 5.2.*

Proof sketch. The graph signature records the value of every binary edge generator. Equality of signatures is an equivalence relation, and its classes are exactly the nonempty common level sets of those generators. Adding a new edge appends one binary coordinate to the signature, so each old atom is either left intact or cut by the two events $g^{-1}(1)$ and $g^{-1}(0)$. □

Theorem 8.9 (Hard-DGM measurability, proof sketch for Theorem F.3)**.** *The main-text statement is Theorem 5.3.*

Proof sketch. Edge eligibility is determined by finitely many edge-bit events, so it is measurable with respect to $\mathcal{F}_t^{\text{edge}}$. The anchor-order sigma-algebra generated by all pairwise anchor comparisons determines the nearest anchor in any eligible subset under deterministic tie-breaking. The exact hard selected index is a finite composition of these two finite decisions, so it is measurable with respect to their join. Movable centroids can still be used for the soft retrieval path, but they are not part of this formal information structure. □

Theorem 8.10 (Soft routing hard limit, proof sketch for Theorem F.4)**.** *The main-text statement is Theorem 5.4.*

Proof sketch. For ordinary nearest-prototype routing, every nonnearest candidate has unnormalized weight smaller by at least $\exp(-\delta/T)$, so the total nonnearest mass is bounded by a finite candidate-count factor times this exponential. For a structural edge gate with uniformly positive weight, a wrong-side signed margin $-\delta_e$ contributes a log-sigmoid term bounded by $-\delta_e/\tau_e$. Degree normalization rescales the exponent but does not remove exponential decay under a uniform degree bound. After normalizing against an eligible candidate with compatibility bounded below and bounded distance-score ratio, the wrong-side mass vanishes as the edge temperature tends to zero. □

Theorem 8.11 (Refine-or-repair descent, proof sketch for Theorem F.5)**.** *The main-text statement is Theorem 5.6.*

Proof sketch. Monotone repair changes only local memories and retrieval centroids while keeping anchors and edges fixed, and by definition does not increase the empirical loss on the fixed scoring buffer $\mathcal{B}_t^{\text{score}}$. The prototype and edge penalties therefore stay fixed, so the local objective $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. A refinement that creates one anchored concept and q edges changes $J_{\mathcal{B}_t^{\text{score}}}$ by the empirical-risk decrease on the scoring buffer minus $\lambda_{\text{proto}} + \lambda_{\text{edge}} q$. The acceptance test requires this net decrease to exceed $\tau + 2\epsilon_m$. □

Theorem 8.12 (Safe refinement, proof sketch for Theorem F.6)**.** *The main-text statement is Theorem 5.7.*

Proof sketch. Conditioning on a finite candidate class fixed before scoring, or generated from a proposal buffer independent of the scoring buffer, Hoeffding's inequality bounds the deviation between empirical and population risk for the old state and for each candidate-refined state. A union bound over this finite collection makes the risk bound simultaneous. Each gain is a difference of two risks, so its empirical-population deviation is at most twice the risk deviation. If the empirical gain exceeds cost plus $\tau + 2\epsilon_m$, the population gain exceeds cost plus τ on this high-probability event. □

Theorem 8.13 (Anchor-stable DGM information structure, proof sketch for Theorem F.7)**.** *The main-text statement is Theorem 5.8.*

Proof sketch. Every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is a deterministic function of the two frozen endpoint anchors. A repair step changes neither endpoint anchor, so it changes neither u_e nor b_e. It also leaves every anchor-distance comparison event unchanged. Therefore the anchor-generated DGM information structure is unchanged, even though the retrieval centroids and local memories may be repaired. □

Corollary 8.14 (Finite refinements, proof sketch for Corollary F.8)**.** *The main-text statement is Corollary 5.9.*

Proof sketch. After m accepted refinements a common potential Φ has decreased by at least $m\tau$. Since the potential is nonnegative, $m\tau \leq \Phi(\mathcal{S}_0)$. This argument requires a common potential; changing local scoring buffers do not by themselves give a global finiteness bound. □

Proposition 8.15 (Tree as generalized hard-DGM, proof sketch for Proposition F.9)**.** *The main-text statement is Proposition 5.10.*

Proof sketch. Make one prototype for each tree leaf and attach the root-to-leaf binary tests as hard eligibility constraints. Every input satisfies exactly the path constraints of the leaf reached by the tree, so generalized hard-DGM selects the same memory and returns the same prediction. □

Theorem 8.16 (DGM data structure, proof sketch for Theorem F.10)**.** *The main-text statement is Theorem 5.11.*

Proof sketch. A prediction computes distances from h to N_t prototypes in $\mathbb{R}^d$, costing

$$O(N_t d).$$

It then evaluates Q_k selected incident edge gates and k selected local memories, costing $O(Q_k + kd + C_{\text{local}})$ including sparse routing weights. Repair adds the local repair cost. Creating one anchor-centroid pair stores two vectors in $\mathbb{R}^d$, still $O(d)$. Adding q distinction edges stores q vectors and thresholds, costing

$$O(qd).$$

For embedding-mean memories, each prototype stores $a_j, c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, while each edge stores one distinction vector in $\mathbb{R}^d$. Hence memory is

$$O(N_t(2d + d_e) + |E_t|d).$$

□

Theorem 8.17 (DGM layer cost, proof sketch for Theorem F.11)**.** *The main-text statement is Theorem 6.1.*

Proof sketch. Computing $q = Qh$ costs $O(dd_q)$. Retrieval costs $C_{\text{ret}}(N_t, d_q)$ by definition. Edge-gate evaluation over selected incident edges costs $O(Q_k)$. For each selected prototype, a rank-r adapter computes

$$W_j K h.$$

The matrix-vector product with one local adapter costs $O(dr)$, so k selected adapters cost

$$O(kdr).$$

The detached write path repeats query, retrieval, and edge-gate evaluation, then updates only selected local states. A rank-one local repair or low-rank sufficient-statistic update costs $O(dr)$ per selected memory, with $O(d_q)$ additional prototype bookkeeping. Hence write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_k + k(d_q + dr)).$$

Because the write path is detached, backpropagation does not need intermediate copies of $\mathcal{S}_t$ for all t. It stores the selected read computation used in the forward loss. □

Theorem 8.18 (One-hot memory, proof sketch for Theorem F.12)**.** *The main-text statement is Theorem 7.1.*

Proof sketch. If $\omega, \omega' \in B_i$, then both one-hot features equal e_i, so

$$W\phi_{\mathcal{P}}(\omega) = We_i = W\phi_{\mathcal{P}}(\omega').$$

Thus $W\phi_{\mathcal{P}}$ is constant on atoms. Conversely, if f is constant on each B_i, put that constant value in column i of W. Then

$$f(\omega) = W\phi_{\mathcal{P}}(\omega)$$

for every ω. □

Theorem 8.19 (CPM conflict, proof sketch for Theorem F.13)**.** *The main-text statement is Theorem 7.2.*

Proof sketch. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

After committing $We_i = v(\omega)$, the later evidence $We_i = v(\omega')$ has the same key. Its innovation relative to the old key span is

$$z = (I - e_i e_i^\top)e_i = 0.$$

Its residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega) \neq 0.$$

□

9 Computational Checks

The theory above is structural, but the model is meant to be executable. We include small reproducible computational checks. These are not meant to settle broad benchmark performance; they test whether refine-or-repair behavior appears in controlled settings. The checks include backpropagation, cache and nearest-neighbor memory, nearest-centroid memory, hidden-regime sequence prediction, and ridge-style local adapters where they are directly comparable. Stronger empirical claims would still require a broader suite with online trees, LVQ, RLS, passive-aggressive updates, replay-buffer SGD, TTT-style adaptation, fast-weight or linear-attention memory, and stronger frozen-encoder baselines. The accompanying script is in the experiment directory and uses PyTorch for the backpropagation baselines and NumPy for the distinction-memory methods.

The first task is a two-context XOR stream. In the first context the label is XOR of two Gaussian cluster signs; in the second context the labels are flipped. The context coordinate is observable, but the learner must discover the relevant distinction. We compare three learners. The offline MLP is trained by ordinary backpropagation on the full stream for 500 epochs and should be read as a batch upper bound, not as a runtime learner. The online MLP predicts each example and then performs one backpropagation step on that example. The DGM-style distinction memory uses the same online protocol: predict first, then observe and refine-or-repair. Results are averaged over five seeds with 100 examples per context.

Model	Runtime update	Prequential accuracy	Test accuracy
Backprop MLP, offline	batch backprop	–	0.993 ± 0.002
Backprop MLP, online	one backprop step	0.569 ± 0.065	0.654 ± 0.038
DGM-style memory, online	refine-or-repair	0.804 ± 0.025	0.944 ± 0.026

Table 1: DGM-style distinction memory versus backpropagation on a limited online contextual-XOR stream. Offline backprop is a batch upper bound. The relevant runtime comparison is between online backpropagation and online distinction refinement. The distinction-memory run has monotone local scoring-objective decrease at accepted refinements in all runs.

Model	Accepted splits	Prequential accuracy	Test accuracy
Repair only	0	0.491	0.501
Refine+repair	9	0.939	0.967

Table 2: Single-run hard-routing ablation on XOR Gaussian clusters. This checks that local repair alone cannot solve a missing-distinction task, while accepted refinements decrease the local penalized objective and recover the required partition.

As a minimal ablation of the refine step, we also run the same distinction-tree hard-routing special case on XOR Gaussian clusters with repair disabled from creating distinctions. The repair-only model has one leaf and cannot represent XOR, while refine+repair creates accepted distinctions with objective decrease.

The most direct check of the runtime-learning claim is a hidden-regime online sequence task. A hidden regime chooses the next-token transition map. The learner is not given the regime label; it observes only a noisy continuous cue and the current token, predicts the next token, and then receives the next-token consequence. This setting separates cache-like repair from missing-distinction refinement: the same current token can require different next tokens in different regimes.

We also run small online image-stream checks on MNIST and CIFAR-10 using cached local datasets. These are deliberately reported as sanity checks, not as evidence that DGM is a competitive image classifier. In these experiments the distinction-memory method uses exemplar/prototype repair. The online backpropagation baseline sees each example once and performs one gradient step. Offline backpropagation is again a batch upper bound.

These experiments test a specific thesis rather than a broad benchmark claim. When the representation lacks a useful distinction, one gradient step on the current example is a weak runtime update. A distinction-memory system can instead create or retrieve a local concept and then repair local state. With full offline access and many epochs, backpropagation remains strong, as expected. The image-stream results should mainly be read as implementation checks; the more relevant settings are context switching, few-shot adaptation with a frozen encoder, long-document memory, repeated syntax regimes, and hidden task identity.

For a stronger adapter benchmark, we train a shared convolutional encoder and freeze it. The adaptation problem is then deliberately narrow: given only five labeled support examples per class, adapt the readout. We compare nearest-neighbor memory, DGM-Ridge as a closed-form projection adapter from Section 6, a one-pass backpropagation head, and a 200-step backpropagation head on the same frozen embeddings. This isolates the large-model use case: a strong backbone already exists, and the question is whether runtime adaptation should be a gradient loop, a nonparametric memory, or a closed-form local repair.

Model	Runtime state	Prequential accuracy	Held-out accuracy
Cache LM	token counts	0.362 ± 0.029	0.285 ± 0.049
Budgeted kNN-LM	64 examples	0.583 ± 0.010	0.346 ± 0.106
Nearest-centroid memory	16 centroids	0.278 ± 0.035	0.235 ± 0.024
DGM repair only	one local memory	0.079 ± 0.017	0.055 ± 0.015
DGM refine+repair	graph memory	$\mathbf{0.617 \pm 0.040}$	$\mathbf{0.680 \pm 0.065}$
Backprop MLP, online	one backprop step	0.292 ± 0.020	0.282 ± 0.086

Table 3: Hidden-regime online sequence prediction, averaged over five seeds. The useful operation is to create distinctions when identical tokens have regime-dependent consequences. The refine+repair run had monotone local scoring-objective decrease at accepted refinements in all runs.

Dataset	Offline backprop	Online backprop	Online DGM-style memory
MNIST	0.843 ± 0.009	0.682 ± 0.038	0.851 ± 0.003
CIFAR-10	0.273 ± 0.003	0.137 ± 0.010	0.243 ± 0.008

Table 4: Image-stream checks. The point is not to beat fully trained image models, but to compare one-pass runtime updates. Prototype/exemplar distinction memory beats one-step online backpropagation in both streams and is competitive with the small offline MLP on MNIST.

Finally, we include a synthetic language-modeling stream. Each document context induces a different next-token transition map. The learner observes a prefix representation and must predict the next token before seeing it. After the next token is revealed, DGM-style memory stores the prefix type and next-token consequence; online backpropagation again receives one gradient step per token.

We also verify the projection-memory residual contraction identity used by local repair. For random W, k, v and step size η, the soft Projection Memory update satisfies

$$v - W_{t+1}k = \frac{1}{1 + \eta \|k\|_2^2}(v - W_t k).$$

Over 200 random trials, the maximum absolute error between the measured residual ratio and the formula was $3.33 \cdot 10^{-16}$.

10 Conclusion

Distinction Graph Machines turn the distinction principle into an explicit runtime learner. The model has prototypes, local memories, distinction edges, repair rules, and a train/test API. Its guiding rule is simple: refine when the concepts are wrong, repair when the local state is wrong.

The theory explains why this separation matters. Contradictions inside an atom cannot be solved by optimizing a measurable predictor. They require refinement. Once the local concept is adequate, its state can be repaired by sufficient-statistic, ridge, or projection-memory updates. DGM turns that separation into a concrete runtime state: prototypes hold local memories, and edges generate the partition and gate the read path.

The resulting picture places gradient learning, projection repair, and distinction refinement in one hierarchy. Backpropagation moves slow parameters; projection memory repairs fast state; DGM adds the missing operation of changing the information structure on which those states are

Dataset	Method	Accuracy	Adapter time
MNIST, 5-shot	one-pass backprop head	0.100 ± 0.000	0.0002s
MNIST, 5-shot	200-step backprop head	0.332 ± 0.009	0.0320s
MNIST, 5-shot	kNN memory	0.486 ± 0.005	0.0006s
MNIST, 5-shot	DGM-Ridge	$\mathbf{0.623 \pm 0.003}$	0.0018s
CIFAR-10, 5-shot	one-pass backprop head	0.115 ± 0.014	0.0003s
CIFAR-10, 5-shot	200-step backprop head	0.359 ± 0.009	0.0325s
CIFAR-10, 5-shot	kNN memory	0.355 ± 0.005	0.0006s
CIFAR-10, 5-shot	DGM-Ridge	$\mathbf{0.397 \pm 0.010}$	0.0004s

Table 5: Few-shot adapter benchmark on frozen embeddings. The kNN memory row is the non-parametric retrieval baseline on the same support examples. DGM-Ridge fits the adapter by a closed-form projection solve. It is substantially faster than the tuned backpropagation head and achieves higher accuracy in this local run.

Task	Offline backprop	Online backprop	Online DGM-style memory
Synthetic LM	0.931 ± 0.071	0.125 ± 0.021	0.912 ± 0.077

Table 6: Synthetic next-token stream. The task is designed to isolate runtime adaptation: the hidden context determines the token transition rule, so the useful operation is to remember a local prefix-to-token consequence after observing it. DGM-style memory approaches the offline batch MLP while one-step online backpropagation adapts slowly.

defined. In language modeling, this means that a runtime learner need not backpropagate through the entire model at inference time: it can maintain hidden-state prototypes, repair next-token memories, and create new distinctions when the stream reveals a genuinely new context.

References

[AW01] Katy S. Azoury and Manfred K. Warmuth. Relative loss bounds for on-line density estimation with the exponential family of distributions. *Machine Learning*, 43:211–246, 2001.

[BHM+16] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[Bre67] Lev M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.

[CDK+06] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

[DH00] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80, 2000.

[GCJ17] Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

[Haz16] Elad Hazan. *Introduction to Online Convex Optimization*. Foundations and Trends in Optimization, 2016.

[KLJ+20] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[Koh90] Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.

[LSR+19] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[MS91] Thomas M. Martinetz and Klaus J. Schulten. A neural-gas network learns topologies. In *Artificial Neural Networks*, pages 397–402, 1991.

[RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[RRD+16] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2016.

[SLD+25] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos

Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[SMM+17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[SS12] Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in Machine Learning*, 4(2):107–194, 2012.

[SWL+20] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

[Vov97] Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing Systems*, volume 10, 1997.

[WCB15] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

[Zin03] Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th International Conference on Machine Learning*, pages 928–936, 2003.

Appendix

A Appendix Roadmap

The appendix follows the same story as the main text. It first states the limitations of the refinement view, then situates the paper relative to prior work, and then gives formal statements and proofs.

- **Part 1.** Appendix B clarifies what the theory proves and what it leaves to modeling, statistics, and experiments.
- **Part 2.** Appendix C places the paper relative to prototype methods, decision trees, information theory, sigma-algebra refinement, backpropagation, projection learning, and test-time adaptation.
- **Part 3.** Appendix D proves the minimal refinement and contradiction theorems.
- **Part 4.** Appendix E proves monotonicity of Bayes risk and the exact log-loss and squared-loss refinement identities.
- **Part 5.** Appendix F proves the algorithmic and bridge guarantees: graph-induced partitions, edge-gated hard limits, refine-or-repair objective descent, tree-as-generalized-hard-DGM equivalence, DGM data-structure complexity, DGM layer cost, and the bridge between partition refinement and CPM conflict.

B Limitations

The theory identifies when a distinction is necessary or valuable at the population-information level. It does not solve the search problem of finding the best distinction in a rich hypothesis class. The online algorithm is greedy and local; its refinement rule concerns local prototypes and their nearby conflicts, not global optimality over all information structures.

The risk identities assume correctly estimated conditional distributions or conditional means. With finite data, refinement can increase estimation error, so useful population distinctions may be statistically unsafe. Complexity penalties, thresholds, validation, Bayesian priors, or minimum leaf sizes are needed in practical systems.

The DGM method specifies one canonical refinement mechanism, based on prototypes and local distinction edges, but this is not claimed to be optimal. Rich domains may require learned prototype creation rules, approximate nearest-neighbor indices, nonlinear distinction edges, and better merge or forgetting policies than the default usage-decayed budget rule in Section 6. The bridge to Projection Learning and CPM is exact for one-hot finite partition memories. Continuous neural representations need additional modeling choices: when a continuous conflict should create a new prototype, how local projection memories should be inherited, and how refined information structures interact with differentiable backbones.

The large-model layer should be read as an architectural path, not as a completed scaling theorem. It gives a trainable read path, a detached write path, and a default budget policy with explicit complexity, but it does not prove optimization guarantees for deep nonconvex backbones, nor does it solve distributed systems issues such as load balancing or stale prototype shards. Budget maintenance is also outside the formal refinement theorem: merge, eviction, and compression may discard distinctions, so their effect must be evaluated as a resource-management approximation.

The experiments are computational checks, not a benchmark suite. A complete empirical evaluation should include stronger online and nonparametric baselines, ablations of repair-only, refine-only, edge-free, and edge-gated variants, and tasks where hidden context changes force new distinctions.

C Related Work

Prototype and memory methods. Prototype and memory methods represent data by stored exemplars or centers, including nearest-neighbor classifiers, LVQ, SOMs, and neural gas [Koh90, MS91]. DGM uses the same computational intuition but adds a refinement rule and an information-structure semantics. It is not merely a prototype model: prototypes define runtime concepts, local memories repair within concepts, and distinction edges are consequence-forced binary generators that both define atoms and gate routing.

Decision trees and splitting criteria. Classical decision trees use information gain and variance reduction as splitting criteria, and online decision trees such as Hoeffding trees make greedy streaming split decisions [DH00]. In our view, a decision tree is a hierarchical hard-routing special case of a generalized distinction graph with arbitrary prototype-level eligibility constraints. It stores distinctions in a rooted hierarchy rather than as local pairwise edges between concepts.

Sparse experts and retrieval layers. Mixture-of-experts and retrieval-augmented models scale neural computation by selecting a small subset of parameters or memories for each input [SMM+17, LSR+19]. A DGM layer has a similar sparse read path, but the selected objects are runtime concepts with local memories, edge-generated information structure, and a detached write path that can refine or repair after observing consequences.

Cache and nonparametric language models. Cache language models and kNN language models improve next-token prediction by retrieving recent or datastore examples [GCJ17, KLJ+20]. DGM-Mean is close in spirit to this family, but its memory is organized by graph-induced concepts rather than by an undifferentiated datastore, and surprising consequences can create new distinctions rather than only update retrieval statistics.

Memory-augmented networks. Memory networks, fast weights, and dynamically expanded architectures add explicit state or capacity to neural models [WCB15, BHM+16, RRD+16]. DGM's distinction is the refine-or-repair decomposition: expansion is accepted only when it decreases a penalized local objective, while ordinary state error is handled by local repair.

Information theory and statistical decision theory. The log-loss identity is the standard relation between optimal code length and conditional entropy, while the squared-loss identity is the orthogonal decomposition of conditional expectations in L^2. Our contribution is to place these identities inside a runtime-learning story: the value of learning a distinction is its reduction in Bayes risk.

Backpropagation and gradient learning. Backpropagation supplies differentiable credit assignment for fixed computation graphs [RHW86]. Distinction Learning addresses a different failure mode: the current information structure may not contain the distinctions needed for any measurable predictor to succeed.

Projection learning and online repair. Bregman projection, mirror descent, proximal methods, passive-aggressive updates, and online ridge-style methods provide minimal-change state updates [Bre67, Zin03, CDK+06, SS12, Haz16, Vov97, AW01]. Conservative Projection Memory uses this idea for runtime associative repair. The present paper demotes projection repair to the continuous-state layer that operates after the relevant information structure has been fixed.

Test-time adaptation. Test-time training adapts parameters using auxiliary losses at inference time [SWL+20]. TTT layers treat hidden states as trainable models updated on test sequences [SLD+25]. Distinction Learning suggests a complementary diagnostic: when runtime evidence conflicts inside an indistinguishable atom, the issue is not only how to update fast state, but what distinction the state failed to represent.

D Minimal Refinement Proofs

Theorem D.1 (Minimal finite refinement, formal version of Theorem 3.1)**.** *Let Ω be finite, let $\mathcal{P}$ be a partition of Ω, and let $E \subseteq \Omega$. Define*

$$\mathcal{P}_E = \{B \cap E : B \in \mathcal{P}, B \cap E \neq \varnothing\} \cup \{B \cap E^c : B \in \mathcal{P}, B \cap E^c \neq \varnothing\}.$$

Then:

- **Part 1.** *$\mathcal{P}_E$ is a partition of Ω.*
- **Part 2.** *$\mathcal{P}_E$ refines $\mathcal{P}$.*
- **Part 3.** *E is $\mathcal{P}_E$-measurable.*
- **Part 4.** *If $\mathcal{Q}$ refines $\mathcal{P}$ and E is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_E$.*

Consequently, $\mathcal{P}_E$ is the unique minimal refinement of $\mathcal{P}$ making E measurable.

Proof. For each $B \in \mathcal{P}$, the nonempty sets among $B \cap E$ and $B \cap E^c$ are disjoint and have union B. Since the atoms of $\mathcal{P}$ are disjoint and cover Ω, the atoms of $\mathcal{P}_E$ are disjoint and cover Ω. Thus Part 1 holds.

Every atom of $\mathcal{P}_E$ is contained in some $B \in \mathcal{P}$, so Part 2 holds. Also

$$E = \bigcup_{B \in \mathcal{P}} (B \cap E),$$

where empty intersections are ignored. Hence E is a union of atoms of $\mathcal{P}_E$, proving Part 3.

Let $\mathcal{Q}$ refine $\mathcal{P}$ and make E measurable. For any atom $Q \in \mathcal{Q}$, there exists $B \in \mathcal{P}$ with

$$Q \subseteq B.$$

Since E is $\mathcal{Q}$-measurable, each atom of $\mathcal{Q}$ is either contained in E or disjoint from E. Hence

$$Q \subseteq B \cap E \qquad \text{or} \qquad Q \subseteq B \cap E^c.$$

Thus each atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}_E$, so $\mathcal{Q}$ refines $\mathcal{P}_E$. This proves Part 4.

If another partition has the same minimal property, it refines $\mathcal{P}_E$ and is refined by $\mathcal{P}_E$, so the two partitions have the same atoms. □

Theorem D.2 (Minimal sigma-algebra refinement, formal version of Theorem 3.2)**.** *Let $(\Omega, \mathcal{A})$ be a measurable space, let $\mathcal{F} \subseteq \mathcal{A}$ be a sigma-algebra, and let $Z : \Omega \to \mathcal{Z}$ be a measurable map into a measurable space $(\mathcal{Z}, \mathcal{B})$. The join $\mathcal{F} \vee \sigma(Z)$ is the smallest sigma-algebra $\mathcal{G} \subseteq \mathcal{A}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable.*

Proof. By definition,

$$\mathcal{F} \vee \sigma(Z) = \sigma(\mathcal{F} \cup \sigma(Z)).$$

Therefore $\mathcal{F} \subseteq \mathcal{F} \vee \sigma(Z)$ and $\sigma(Z) \subseteq \mathcal{F} \vee \sigma(Z)$, so Z is measurable with respect to $\mathcal{F} \vee \sigma(Z)$.

Let $\mathcal{G}$ be any sigma-algebra such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable. The latter condition means $\sigma(Z) \subseteq \mathcal{G}$. Hence

$$\mathcal{F} \cup \sigma(Z) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(Z)) \subseteq \mathcal{G}.$$

Thus $\mathcal{F} \vee \sigma(Z) \subseteq \mathcal{G}$. □

Theorem D.3 (Contradiction requires distinction, formal version of Theorem 3.3)**.** *Let $\mathcal{P}$ be a partition of Ω and let $y : \Omega \to A$ be a required output. Suppose there exist $\omega, \omega' \in B$ for some atom $B \in \mathcal{P}$ such that $y(\omega) \neq y(\omega')$. Then no $\mathcal{P}$-measurable $f : \Omega \to A$ satisfies both*

$$f(\omega) = y(\omega), \qquad f(\omega') = y(\omega').$$

Proof. Since f is $\mathcal{P}$-measurable and $\omega, \omega' \in B$,

$$f(\omega) = f(\omega').$$

If both requirements held, then

$$y(\omega) = f(\omega) = f(\omega') = y(\omega'),$$

contradicting $y(\omega) \neq y(\omega')$. □

E Value of Distinction Proofs

Theorem E.1 (Bayes-risk monotonicity, formal version of Theorem 4.1)**.** *Let Y be a target random variable, let A be an action space, and let L be an integrable loss. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

Proof. Define

$$\mathcal{D}_{\mathcal{F}} = \{a : \Omega \to A : a \text{ is } \mathcal{F}\text{-measurable}\}, \qquad \mathcal{D}_{\mathcal{G}} = \{a : \Omega \to A : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Since $\mathcal{F} \subseteq \mathcal{G}$,

$$\mathcal{D}_{\mathcal{F}} \subseteq \mathcal{D}_{\mathcal{G}}.$$

Therefore

$$\begin{aligned} R_L(\mathcal{G}) &= \inf_{a \in \mathcal{D}_{\mathcal{G}}} \mathbb{E}[L(Y,a)] \\ &\leq \inf_{a \in \mathcal{D}_{\mathcal{F}}} \mathbb{E}[L(Y,a)] \\ &= R_L(\mathcal{F}). \end{aligned}$$

□

Theorem E.2 (Log-loss value, formal version of Theorem 4.2)**.** *Let Y be finite and let $\mathcal{F} \subseteq \mathcal{G}$. Under log loss, an admissible action is a measurable conditional distribution $q(\cdot)$ on the range of Y, and the loss is $-\log q(Y)$. Then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Proof. For any $\mathcal{F}$-measurable conditional distribution q, write $p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F})$. Then

$$\begin{aligned} \mathbb{E}[-\log q(Y) \mid \mathcal{F}] &= \sum_y p_{\mathcal{F}}(y)(-\log q(y)) \\ &= \sum_y p_{\mathcal{F}}(y)(-\log p_{\mathcal{F}}(y)) + \sum_y p_{\mathcal{F}}(y) \log \frac{p_{\mathcal{F}}(y)}{q(y)} \\ &= H(Y \mid \mathcal{F}) + D_{\mathrm{KL}}(p_{\mathcal{F}} \| q). \end{aligned}$$

Taking expectation and minimizing over q gives

$$R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F}).$$

Likewise,

$$R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G}).$$

By definition of conditional mutual information between Y and the additional information $\mathcal{G}$ given $\mathcal{F}$,

$$I(Y; \mathcal{G} \mid \mathcal{F}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).$$

Combining proves the claim. □

Theorem E.3 (Squared-loss value, formal version of Theorem 4.3)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y,a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

Proof. The optimal $\mathcal{F}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, and the optimal $\mathcal{G}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{G}]$. Let

$$U = Y - \mathbb{E}[Y \mid \mathcal{G}], \qquad V = \mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}].$$

Then

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = U + V.$$

Since V is $\mathcal{G}$-measurable and $\mathbb{E}[U \mid \mathcal{G}] = 0$,

$$\mathbb{E}[\langle U, V\rangle \mid \mathcal{G}] = \langle \mathbb{E}[U \mid \mathcal{G}], V\rangle = 0.$$

Thus

$$\begin{aligned}\mathbb{E}\,\|Y - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2 &= \mathbb{E}\,\|U + V\|_2^2 \\ &= \mathbb{E}\,\|U\|_2^2 + \mathbb{E}\,\|V\|_2^2 \\ &= R_2(\mathcal{G}) + \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].\end{aligned}$$

Rearranging proves the identity. □

Proposition E.4 (Finite squared-loss split identity)**.** *Let $B = B_0 \cup B_1$ be a disjoint split of a finite sample set. With empirical means $\hat{\mu}_B, \hat{\mu}_{B_0}, \hat{\mu}_{B_1}$ and* SSE *as in Section 4,*

$$\mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1) = |B_0|\,\|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\,\|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

Proof. For each child B_b,

$$\begin{aligned}\sum_{j\in B_b} \|y_j - \hat{\mu}_B\|_2^2 &= \sum_{j\in B_b} \|y_j - \hat{\mu}_{B_b} + \hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \\ &= \sum_{j\in B_b} \|y_j - \hat{\mu}_{B_b}\|_2^2 + |B_b|\,\|\hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2,\end{aligned}$$

because

$$\sum_{j\in B_b} (y_j - \hat{\mu}_{B_b}) = 0.$$

Summing over $b \in \{0, 1\}$ gives the result. □

F DGM Algorithm and Projection-Bridge Proofs

Theorem F.1 (Prototype edge separation, formal version of Theorem 5.1)**.** *Let $a_i, a_j \in \mathbb{R}^d$ with $a_i \neq a_j$. Define*

$$u = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b = \frac{1}{2}u^\top(a_i + a_j).$$

Then

$$u^\top a_i - b = \frac{1}{2}\,\|a_i - a_j\|_2, \qquad u^\top a_j - b = -\frac{1}{2}\,\|a_i - a_j\|_2.$$

Proof. Compute

$$\begin{aligned}u^\top a_i - b &= u^\top a_i - \frac{1}{2}u^\top a_i - \frac{1}{2}u^\top a_j \\ &= \frac{1}{2}u^\top(a_i - a_j) \\ &= \frac{1}{2}\,\|a_i - a_j\|_2.\end{aligned}$$

Similarly,

$$\begin{aligned} u^\top a_j - b &= u^\top a_j - \frac{1}{2}u^\top a_i - \frac{1}{2}u^\top a_j \\ &= -\frac{1}{2}u^\top (a_i - a_j) \\ &= -\frac{1}{2}\left\|a_i - a_j\right\|_2 . \end{aligned}$$

□

Theorem F.2 (DGM graph generates a finite information structure, formal version of Theorem 5.2)**.** *Let E be a finite set of binary generators $g_e : \mathcal{H} \to \{0,1\}$. Define*

$$\sigma_E(h) = \big(g_e(h)\big)_{e\in E}.$$

The relation $h \sim_E h'$ iff $\sigma_E(h) = \sigma_E(h')$ is an equivalence relation. Its nonempty equivalence classes form a finite partition $\mathcal{P}_E^G$ of $\mathcal{H}$. If a new generator g is added, then

$$\mathcal{P}_{E\cup\{g\}}^G = \mathcal{P}_E^G \vee \sigma(g),$$

where the right side denotes cutting each atom of $\mathcal{P}_E^G$ by $g^{-1}(1)$ and $g^{-1}(0)$ and keeping nonempty pieces.

Proof. Equality of signatures is reflexive, symmetric, and transitive, so $\sim_E$ is an equivalence relation. Equivalence classes are disjoint, nonempty, and cover $\mathcal{H}$, hence they form a partition. Since E is finite, there are at most $2^{|E|}$ signatures, so the partition is finite.

For the update, two points have the same signature under $E \cup \{g\}$ exactly when they have the same old E-signature and the same value of g. Therefore each new equivalence class is an old atom intersected with either $g^{-1}(1)$ or $g^{-1}(0)$, with empty intersections discarded. This is precisely $\mathcal{P}_E^G \vee \sigma(g)$. □

Theorem F.3 (Anchor-DGM routing is graph-plus-anchor-order measurable, formal version of Theorem 5.3)**.** *Let E_t be finite and let $a_1, \ldots, a_{N_t} \in \mathbb{R}^d$ be finite frozen anchors with deterministic tie-breaking. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be generated by all pairwise anchor-distance comparison events*

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}.$$

The exact Hard-DGM selected prototype index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.

Proof. For each prototype j, eligibility is an intersection of finitely many edge-bit events of the form

$$\{h : g_e(h) = 1\} \qquad \text{or} \qquad \{h : g_e(h) = 0\},$$

depending on j's recorded side of edge e. Hence the eligible set for j is $\mathcal{F}_t^{\text{edge}}$-measurable.

The anchor-order sigma-algebra determines the complete order of all anchors under the fixed tie-breaking rule. For any subset $A \subseteq \{1, \ldots, N_t\}$, the event that prototype j is the nearest anchor in A is a finite intersection of pairwise anchor-comparison events, hence belongs to $\mathcal{F}_t^{\text{anchor-order}}$. The selected Hard-DGM index is obtained by combining finitely many eligibility events with these nearest-in-subset events. Therefore the inverse image of each selected index is a finite union of intersections of sets from $\mathcal{F}_t^{\text{edge}}$ and $\mathcal{F}_t^{\text{anchor-order}}$, and belongs to their join. □

Theorem F.4 (Soft routing approaches hard routing, formal version of Theorem 5.4)**.** *Let N be a finite candidate set and let*

$$\alpha_j = \frac{\exp(-d_j/T)\chi_j}{\sum_{r\in N}\exp(-d_r/T)\chi_r}.$$

If $j^\star$ is the unique candidate with $d_j - d_{j^\star} \geq \delta > 0$ for all $j \neq j^\star$ and $\chi_j = \chi_{j^\star} = 1$, then

$$1 - \alpha_{j^\star} \leq (|N| - 1)\exp(-\delta/T).$$

Moreover, suppose $\log \chi_j$ contains a term

$$\frac{\beta_e}{\sqrt{D_j}} \log \operatorname{sigmoid}\left(\frac{m_j}{\tau_e}\right)$$

with $\beta_e > 0$ and finite $D_j \geq 1$, and candidate j violates that edge with $m_j \leq -\delta_e < 0$. Then $\chi_j \leq \exp(-\beta_e\delta_e/(\sqrt{D_j}\tau_e))$.

For a wrong-side set $A \subseteq N$, suppose every $j \in A$ violates some incident structural edge whose weight is at least $\beta_{\min} > 0$, whose signed margin is at most $-\delta_e$, and whose incident degree is at most $D_{\max}$. If at least one eligible reference candidate r has $\chi_r \geq \chi_{\min} > 0$, and the distance score ratio between every $j \in A$ and r is bounded by C, then the total wrong-side routing mass is at most

$$\frac{C|N|}{\chi_{\min}} \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Proof. For the nearest-prototype statement,

$$\frac{\sum_{j\neq j^\star}\exp(-d_j/T)}{\exp(-d_{j^\star}/T)} = \sum_{j\neq j^\star}\exp(-(d_j - d_{j^\star})/T) \leq (|N| - 1)\exp(-\delta/T).$$

Since

$$1 - \alpha_{j^\star} = \frac{\sum_{j\neq j^\star}\exp(-d_j/T)}{\exp(-d_{j^\star}/T) + \sum_{j\neq j^\star}\exp(-d_j/T)},$$

the same upper bound follows.

For an edge violation, set $a = \delta_e/\tau_e > 0$. Since

$$\operatorname{sigmoid}(-a) = \frac{1}{1+\exp(a)} \leq \exp(-a),$$

the corresponding log-compatibility contribution is at most $-\beta_e\delta_e/(\sqrt{D_j}\tau_e)$, and therefore $\chi_j \leq \exp(-\beta_e\delta_e/(\sqrt{D_j}\tau_e))$. If A is the wrong-side set and r is an eligible candidate whose compatibility satisfies $\chi_r \geq \chi_{\min}$, the uniform wrong-side assumptions give, for every $j \in A$,

$$\chi_j \leq \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Combining this with the assumed distance-score ratio bound gives

$$\begin{aligned}\sum_{j\in A}\alpha_j &\leq \sum_{j\in A}\frac{C}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right)\\ &\leq \frac{C|N|}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).\end{aligned}$$

□

Theorem F.5 (Refine-or-repair objective descent, formal version of Theorem 5.6)**.** *Let*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{proto}} N_{\text{proto}}(\mathcal{S}) + \lambda_{\text{edge}} |E(\mathcal{S})|$$

with nonnegative penalties on a fixed scoring buffer. If a repair step leaves anchors, edges, and prototype count unchanged and is monotone on the scoring buffer $\mathcal{B}_t^{\text{score}}$, then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement creates one prototype and q edges and is accepted only when

$$\Delta_{\text{refine}} > \lambda_{\text{proto}} + \lambda_{\text{edge}} q + \tau + 2\epsilon_m,$$

where

$$\Delta_{\text{refine}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}),$$

then

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) < J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - \tau - 2\epsilon_m.$$

Proof. For repair, N_{proto} and $|E|$ are unchanged. The empirical-risk term on $\mathcal{B}_t^{\text{score}}$ is no larger by assumption, so $J_{\mathcal{B}_t^{\text{score}}}$ does not increase.

For refinement,

$$\begin{aligned} J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) &= \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{proto}} - \lambda_{\text{edge}} q \\ &= \Delta_{\text{refine}} - \lambda_{\text{proto}} - \lambda_{\text{edge}} q \\ &> \tau + 2\epsilon_m. \end{aligned}$$

Rearranging gives the claim. □

The theorem is local to the fixed scoring buffer used in its statement. If the scoring buffer changes over time, these inequalities alone do not define a common global descent potential.

Theorem F.6 (Statistically safe finite-class refinement, formal version of Theorem 5.7)**.** *Assume losses lie in $[0, 1]$. Let $\mathcal{H}$ be a finite class of candidate refinements fixed before scoring, or generated from a proposal buffer independent of a scoring buffer $\mathcal{B}^{\text{score}}$ containing m independent samples. For candidate g, let $\widehat{\Delta}(g)$ be the empirical gain on $\mathcal{B}^{\text{score}}$ and let $\Delta(g)$ be the corresponding population gain. Then, with probability at least $1 - \delta$, every $g \in \mathcal{H}$ satisfies*

$$\left|\widehat{\Delta}(g) - \Delta(g)\right| \le 2\epsilon_m, \qquad \epsilon_m = \sqrt{\frac{\log(2(|\mathcal{H}| + 1)/\delta)}{2m}}.$$

Consequently, if

$$\widehat{\Delta}(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_m,$$

then on the same event

$$\Delta(g) > \lambda \operatorname{cost}(g) + \tau.$$

Proof. Apply Hoeffding's inequality to the empirical risk of the old state and to the empirical risk of each candidate-refined state. For any fixed state in this finite collection,

$$\mathbb{P}\left(\left|\widehat{R} - R\right| > \epsilon\right) \le 2\exp(-2m\epsilon^2).$$

Because $\mathcal{H}$ is fixed before scoring or generated from data independent of $\mathcal{B}^{\text{score}}$, Hoeffding's inequality applies conditionally on $\mathcal{H}$. Taking a union bound over the old state and the $|\mathcal{H}|$ candidate states gives the displayed ϵ_m. On this event, each gain is a difference of two risks, so empirical and population gain differ by at most $2\epsilon_m$. The implication follows by subtracting this deviation from the empirical gain lower bound. □

Theorem F.7 (Anchor-stable DGM information structure, formal version of Theorem 5.8)**.** *Let*

$$\mathcal{F}_t^{\mathrm{DGM}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\} : 1 \le i, j \le N_t\Big).$$

Suppose every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is computed from frozen endpoint anchors a_i, a_j. Any repair step that changes only centroids c_j, local memories M_j, and counts n_j, while keeping anchors and edges fixed, leaves $\mathcal{F}_t^{\mathrm{DGM}}$ unchanged. If a refinement adds a new anchor and new edge generators, then $\mathcal{F}_{t+1}^{\mathrm{DGM}}$ is $\mathcal{F}_t^{\mathrm{DGM}}$ joined with the new anchor-order events and the new edge generators.

Proof. For an edge $e = (i, j)$, both

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j)$$

are deterministic functions of the frozen anchors. A repair step does not change these anchors, so it does not change u_e, b_e, or g_e. It also does not change any anchor-distance comparison event because all anchors are fixed. Hence both sigma-algebras in the join defining $\mathcal{F}_t^{\mathrm{DGM}}$ are unchanged.

If a refinement adds anchor a_{new} and edge generators E_{new}, the new edge sigma-algebra is $\sigma(g_e : e \in E_t) \vee \sigma(g_e : e \in E_{\mathrm{new}})$, and the new anchor-order sigma-algebra is the old anchor-order sigma-algebra joined with the pairwise comparisons involving a_{new}. This is exactly the stated join. □

Corollary F.8 (Finite number of accepted refinements, formal version of Corollary 5.9)**.** *Let Φ be a common potential with $\Phi(\mathcal{S}) \ge 0$ for all reachable states. If each accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$.*

Proof. After m accepted refinements,

$$\Phi(\mathcal{S}_m) \le \Phi(\mathcal{S}_0) - m\tau.$$

Since $\Phi(\mathcal{S}_m) \ge 0$, we have $m \le \Phi(\mathcal{S}_0)/\tau$. □

Proposition F.9 (Trees as generalized hard-DGMs, formal version of Proposition 5.10)**.** *Let $\mathcal{T}$ be a binary decision tree whose internal nodes compute binary distinctions $g_\nu(h) \in \{0, 1\}$ and whose leaves have predictors M_ℓ. Construct a generalized hard-DGM with one prototype c_ℓ and memory M_ℓ for each leaf ℓ, where the generalized model permits arbitrary finite binary eligibility constraints in addition to pairwise prototype edges. For each leaf, add hard eligibility constraints equal to the signed distinctions along the root-to-leaf path. If generalized hard-DGM routes to the unique eligible leaf prototype, then its prediction equals the tree prediction for every h.*

Proof. For any input h, the tree follows exactly one root-to-leaf path. The leaf reached by the tree is the unique leaf whose path constraints are all satisfied by h. By construction, the corresponding generalized hard-DGM prototype has exactly those eligibility constraints, and all other leaf prototypes violate at least one path constraint at the first node where their path differs from the tree path. Therefore generalized hard-DGM selects the same leaf and applies the same memory M_ℓ. □

Theorem F.10 (DGM data-structure bound, formal version of Theorem 5.11)**.** *Assume DGM has N_t centroids in $\mathbb{R}^d$, exact nearest-centroid search, sparse neighborhood size k, selected incident edge-gate count Q_k, and local-memory evaluation cost C_{local}. Then one prediction costs $O(N_t d + Q_k + kd + C_{\mathrm{local}})$. One observation that repairs a selected prototype costs this plus the local repair cost. One observation that creates a new anchor-centroid pair and q distinction edges costs an additional $O(d + qd)$. If each prototype stores an anchor, a centroid, and an embedding mean in $\mathbb{R}^{d_e}$, and each edge stores a linear distinction in $\mathbb{R}^d$, then the runtime state uses $O(N_t(2d + d_e) + |E_t|d)$ memory.*

Proof. A distance computation between $h \in \mathbb{R}^d$ and one prototype costs $O(d)$. Exact search over N_t prototypes therefore costs

$$O(N_t d).$$

After the k nearest centroids are selected, evaluating the incident edge gates costs $O(Q_k)$. Computing sparse routing weights, distances within the selected set, and evaluating selected local memories costs

$$O(kd + C_{\text{local}}).$$

Thus prediction costs

$$O(N_t d + Q_k + kd + C_{\text{local}}).$$

Observation begins with the same retrieval computation. If it repairs a selected prototype, it adds the local repair cost. If it creates a new anchor-centroid pair, storing the pair costs $O(d)$. For each new edge, computing and storing the linear distinction vector costs $O(d)$, so q edges cost

$$O(qd).$$

The total additional cost for refinement is therefore $O(d + qd)$.

For memory, each prototype stores $a_j, c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, giving

$$O(N_t(2d + d_e)).$$

Each edge stores one distinction vector and threshold, giving

$$O(|E_t|d).$$

Adding the two terms proves the claimed bound. □

Theorem F.11 (DGM layer cost, formal version of Theorem 6.1)**.** *Consider a DGM-Adapter layer with hidden dimension d, query dimension d_q, local rank r, top-k retrieval, selected incident edge-gate count Q_k, and prototype-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_k + kdr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_k + k(d_q + dr)).$$

For a block of length L, backpropagation through the slow parameters does not require storing the full DGM state history $\mathcal{S}_1, \ldots, \mathcal{S}_L$.

Proof. For a token hidden state $h \in \mathbb{R}^d$, the query computation

$$q = Qh$$

costs $O(dd_q)$. The retrieval cost is $C_{\text{ret}}(N_t, d_q)$ by definition. Evaluating selected incident edge gates costs $O(Q_k)$.

For a selected prototype j, the rank-r adapter read is

$$W_j K h.$$

Multiplying one selected $W_j \in \mathbb{R}^{d\times r}$ by a rank-r key costs $O(dr)$. Therefore k selected adapters cost

$$O(kdr).$$

The routing weights add lower-order top-k normalization cost, absorbed into the same bound. This proves the read overhead.

The write path first computes the same query, retrieval, and edge gates. It then updates selected prototype statistics and selected local memories. Prototype bookkeeping costs $O(d_q)$ per selected prototype. A rank-one local repair of a rank-r or low-rank sufficient-statistic memory costs $O(dr)$ per selected memory. Hence the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_k + k(d_q + dr)).$$

Finally, the update rule is applied as

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

Thus the computational graph for the loss contains the selected read computation, but not the chain of state-update maps from $\mathcal{S}_1$ through $\mathcal{S}_L$. Therefore full DGM state history is not required for backpropagation through the slow parameters. □

Theorem F.12 (One-hot memory is partition-measurable, formal version of Theorem 7.1)**.** *Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}}(\omega) = e_i$ for $\omega \in B_i$. For every $W \in \mathbb{R}^{m\times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is equal to f_W for some $W \in \mathbb{R}^{m\times n}$.*

Proof. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

Thus

$$f_W(\omega) = We_i = f_W(\omega'),$$

so f_W is constant on each atom and is $\mathcal{P}$-measurable.

Conversely, if f is $\mathcal{P}$-measurable, it is constant on each B_i. Let w_i be this constant value on B_i, and define

$$W = [w_1, \ldots, w_n].$$

Then for $\omega \in B_i$,

$$W\phi_{\mathcal{P}}(\omega) = We_i = w_i = f(\omega).$$

□

Theorem F.13 (CPM conflict equals missing distinction, formal version of Theorem 7.2)**.** *Let* $\mathcal{P} = \{B_1, \ldots, B_n\}$ *and let* $\omega, \omega' \in B_i$*. Suppose* $v(\omega) \neq v(\omega')$*. Let* $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$*. If a memory* W *has already committed* $We_i = v(\omega)$*, then the evidence* $We_i = v(\omega')$ *has innovation* $z = 0$ *relative to the old key span and residual* $e \neq 0$*.*

Proof. Since $\omega, \omega' \in B_i$,

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

The old key span contains e_i. Therefore the innovation of the new key $k = e_i$ relative to the old key matrix $K = e_i$ is

$$z = (I - KK^{\dagger})k = (I - e_i e_i^{\top})e_i = 0.$$

The new residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega).$$

Since $v(\omega') \neq v(\omega)$, we have $e \neq 0$. □